Architecture Decision Record

ADR-001: Process decomposition for customer-initiated account amendments

Field	Value
Status	Proposed
Date	23 July 2026
Decision area	BPMN process structure, jBPM / BAMOE
Supersedes	—
Related	ADR-002 (rules placement), ADR-003 (embedded vs standalone)

1. Context

The programme automates customer-initiated account amendments. Three amendment journeys are in scope for the first release:

- Change of address (CoA)
- Change of name (CoN)
- Conversion of a joint account to a sole account (joint to sole)

A customer may request more than one amendment in a single submission. Each request must present to the customer and to back-office staff as a single case with one status, one reference and one overall SLA.

The journeys are human-centric and long-running: they involve maker-checker review, document collection, external screening and, for joint to sole, consent from multiple account parties. Every state change must be auditable for regulatory purposes.

The engine is jBPM in its BAMOE distribution, running embedded in a Spring Boot service (see ADR-003). Process persistence is enabled so instances survive restart and rescheduling.

2. Problem statement

Two structural questions must be settled before modelling begins:

1. Should the three amendment types be modelled inside one process definition, or as separate definitions invoked by a parent?
2. How should ad-hoc needs — an extra document request, an additional verification step, a customer cancellation — be represented?

An initial proposal was to make the parent case-centric and model all four concerns — the three amendment types plus the ad-hoc needs — as event subprocesses within a single parent definition.

3. Decision

We will model an Amendment Request parent process that acts as the case container. Within it:

- CoA, CoN and joint to sole are invoked as call activities, each resolving to its own independently deployable process definition.
- Ad-hoc, event-triggered concerns are modelled as event subprocesses attached to the parent.
- Ad-hoc concerns specific to a single amendment type are modelled as event subprocesses inside that child process, not hoisted to the parent.

We will not model the three amendment types as event subprocesses.

3.1 Element allocation

Concern	Construct	Rationale
Amendment request	Parent process	Case container; owns case file, correlation id, overall SLA
Intake, screening, eligibility	Tasks in parent	Shared across all amendment types
CoA	Call activity	Independent version and deploy cadence
CoN	Call activity	Independent version and deploy cadence
Joint to sole	Call activity	Highest risk and change rate; benefits most from isolation
Extra document request	Event subprocess, non-interrupting	Event-triggered; main flow continues
Additional verification	Event subprocess, non-interrupting	Event-triggered; main flow continues
Cancellation or withdrawal	Event subprocess, interrupting	Terminates the request
SLA escalation	Event subprocess or boundary timer	Time-triggered, cross-cutting
Party disputes consent	Event subprocess in joint to sole child	Specific to one amendment type

4. Rationale

4.1 Why event subprocesses are wrong for the amendment types

An event subprocess is triggered by an event and is intended for cross-cutting concerns that may fire at unpredictable points. Its semantics are reactive: something happened, respond to it.

The three amendment types are not reactions to events. They are the primary work, selected at intake by what the customer requested. Modelling them as event subprocesses would produce four specific defects:

- Synthetic triggers. A signal such as "CoA requested" would have to be fabricated purely to start work that is already known at intake.
- Ambiguous concurrency. Where a customer requests CoA and CoN together, two non-interrupting event subprocesses would run alongside the parent main flow with no clear join semantics.
- Coupled deployment. All three journeys would share one process definition. A change to joint to sole would force redeployment of CoA and CoN.
- Degraded observability. Instance-level metrics and SLA measurement would be per parent instance rather than per amendment type.

4.2 Why call activities are correct

A call activity invokes a separate process definition. This preserves the case-centric parent while giving each amendment type an independent lifecycle:

- Each journey is versioned, tested and deployed independently. Joint to sole carries the greatest regulatory scrutiny and the highest expected change rate, so isolating it is the highest-value separation.
- Each journey can carry its own DMN and rule assets without redeploying the others.
- Per-type SLA, throughput and failure metrics are available at instance level.
- Multi-amendment submissions are handled by a multi-instance call activity, sequential or parallel per business rule.

4.3 Why event subprocesses are correct for ad-hoc concerns

Extra document requests, additional verification, cancellation and SLA escalation are genuinely event-triggered and can occur at unpredictable points in the journey. Attaching them to the parent means they apply regardless of which child process is currently active, which is precisely the adaptive behaviour required at case level.

Interrupting semantics are used only where the event terminates the request, as with customer cancellation. All other ad-hoc handlers are non-interrupting so the main flow continues.

5. Consequences

5.1 Positive

- The customer and back-office see a single case: one reference, one status, one case file, one overall SLA.
- The three journeys evolve independently, reducing regression surface and release coordination.
- Foreseeable exceptions are handled without dynamic case fragments.
- Ad-hoc handling nests naturally: case-level concerns on the parent, type-specific concerns in the child.

5.2 Negative and accepted

- Four process definitions to maintain rather than one, with a defined parent-to-child data contract.
- Correlation and case-file propagation between parent and children must be explicitly designed.
- Cross-instance visibility spanning parent and children must be assembled through the data index and event layer rather than read from a single definition.

5.3 Explicitly out of scope

This design delivers a structured process with case-like framing. It does not deliver adaptive case management. Knowledge workers cannot create arbitrary new tasks at runtime, and there is no bundled case portal.

If the business requires caseworkers to improvise the path, this decision must be revisited and a case management platform evaluated instead. This is the single largest fit risk in the design and should be confirmed with the business before build starts.

6. Alternatives considered

Alternative	Assessment
Amendment types as event subprocesses in one parent	Rejected. Requires synthetic triggers, couples deployment of all three journeys, and produces ambiguous concurrency for multi-amendment requests.
One process definition per amendment type, no parent	Rejected. Loses the single case view for multi-amendment submissions; duplicates intake, screening and eligibility across three definitions.
Fully adaptive case with dynamic task creation	Rejected for this release. Case support in the current line is less mature than the legacy offering and no case portal is bundled; the journeys are largely predictable, so the flexibility is not required.
Embedded subprocesses instead of call activities	Rejected. Retains the coupled deployment and versioning problem while adding no benefit over call activities.

7. Open items

Item	Owner	Needed by
Confirm business does not require adaptive case management	Business analysis	Before build start
Confirm current BAMOE release and support lifecycle dates	Architecture	Before version lock
Confirm state of case management support in the 9.x line	Architecture	Before build start
Define parent-to-child data contract and correlation strategy	Design	Sprint 1
Decide rule change cadence: in-process DMN or separate	Architecture	Sprint 1

Item	Owner	Needed by
decision service		

Two items above are version-sensitive and should be verified against current vendor documentation before this ADR is accepted.